Instituto J&F
CENTRO DE EDUCAÇÃO PARA OS NEGÓCIOS

LAB – DevOps – Pipeline CI/CD

Aluno: Noé Carrion | **DevOps–LAB** | **Out/2026**

Enviar em formato PDF no link: https://forms.cloud.microsoft/r/Gbb4U2yXbm

Atividade — Pipeline CI/CD com IA Generativa

1. Informações do Projeto Escolhido

Projeto / Aplicação:	**Flop+ (FlopPlus)** — Streaming Web App
Repositório no GitHub:	https://github.com/noecarrion/FlopPlus
Branch Criada e Enviada:	`feature/ci-cd-pipeline`
Pull Request no GitHub:	https://github.com/noecarrion/FlopPlus/pull/new/feature/ci-cd-pipeline
Aplicação Publicada (CD):	https://noecarrion.github.io/FlopPlus/

2. Escopo do Deployment e Justificativa da Estratégia

O projeto escolhido, **Flop+**, é uma aplicação web desenvolvida em **HTML5** e **CSS3** com elementos multimídia (FontAwesome, imagens AVIF/PNG), tratando-se de uma aplicação cliente puramente estática.

A estratégia de deployment adotada foi o **GitHub Pages automatizado via GitHub Actions**, pelas seguintes razões técnicas:

- **Adequação Tecnológica:** Aplicações estáticas front-end não demandam servidores de aplicação (Node/Python) nem contêineres Docker em execução contínua. Utilizar plataformas como Render ou Railway acarretaria complexidade e alocação desnecessária de recursos;
- **Distribuição Global e Alta Disponibilidade:** O GitHub Pages provê CDN global integrada de alta velocidade e certificados SSL/TLS automáticos;
- **Segurança com OIDC Nativo:** O deploy utiliza as actions oficiais do GitHub (`actions/deploy-pages@v4` e `actions/upload-pages-artifact@v3`) através de permissões nativas de token do repositório (`pages: write` e `id-token: write`), eliminando a necessidade de expor chaves secretas ou credenciais externas de terceiros;
- **Garantia de Qualidade (Quality Gate):** O fluxo assegura que o deployment ocorra somente em push na branch `main` e se as etapas anteriores de CI (validação e testes) forem concluídas com sucesso.

3. Arquivo da Pipeline (.github/workflows/ci-cd.yml)

A pipeline foi implementada no arquivo `.github/workflows/ci-cd.yml` contendo dois jobs sequenciais e interdependentes:

```yaml
name: Pipeline CI/CD - FlopPlus

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  workflow_dispatch:

# Permissões necessárias para publicação no GitHub Pages via OIDC
permissions:
  contents: read
  pages: write
  id-token: write

# Permite apenas uma implantação concorrente no Pages
concurrency:
  group: 'pages'
  cancel-in-progress: false
```

```
jobs:
  # ============================================================
  # ETAPA 1: CONTINUOUS INTEGRATION (CI)
  # Executada a cada push e pull_request direcionados a main
  # ============================================================
  continuous-integration:
    name: CI - Build, Validacao e Testes
    runs-on: ubuntu-latest

    steps:
      - name: 1. Checkout do Repositorio
        uses: actions/checkout@v4

      - name: 2. Configurar Ambiente Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: 3. Instalar Dependencias
        run: npm ci

      - name: 4. Executar Testes de Integridade de Links e Assets
        run: npm test

      - name: 5. Executar Validacao Sintatica HTML (HTMLHint)
        run: npm run lint:ci

  # ============================================================
  # ETAPA 2: CONTINUOUS DEPLOYMENT (CD)
  # Executada apenas em push na main, apos conclusao da CI
  # ============================================================
  continuous-deployment:
    name: CD - Deploy no GitHub Pages
    needs: continuous-integration
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest

    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}

    steps:
      - name: 1. Checkout do Repositorio
        uses: actions/checkout@v4

      - name: 2. Configurar GitHub Pages
        uses: actions/configure-pages@v5

      - name: 3. Fazer Upload do Artefato do Site
        uses: actions/upload-pages-artifact@v3
        with:
          path: '.'

      - name: 4. Realizar Deploy no GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

4. Registro do Processo com IA Generativa

Durante o desenvolvimento da pipeline, a IA Generativa foi utilizada como ferramenta de apoio em cada etapa de tomada de decisão, diagnóstico de falhas e automação. Abaixo estão os prompts reais empregados com linguagem simples, direta e natural, acompanhados da evolução do projeto:

Interação 1 — Definindo a estrutura da pipeline

Prompt utilizado: *"Oi! Preciso criar uma pipeline de CI/CD com GitHub Actions pro meu projeto da faculdade (Flop+). É um site em HTML e CSS no GitHub. O que eu preciso colocar no workflow pra testar e publicar ele no GitHub Pages?"*

Resultado obtido: A IA explicou os conceitos e sugeriu dividir o fluxo em duas etapas: um job de CI para checar os arquivos e um job de CD para publicar automaticamente no GitHub Pages usando as actions oficiais da plataforma.

Problema encontrado: O repositório original não tinha arquivo `package.json`, testes nem nenhum comando configurado para servir de etapa de validação na CI.

Alteração realizada: Inicializamos o projeto com Node.js/npm para viabilizar scripts automatizados e instalamos o linter `htmlhint`.

Interação 2 — Verificando os links do site para o GitHub Pages

Prompt utilizado: *"Eu tenho alguns links nas páginas que estão com barra no começo, tipo href="/index.html" e href="/HTML/aboutme.html". No GitHub Pages isso vai funcionar ou dá problema de link quebrado?"*

Resultado obtido: A IA alertou que no GitHub Pages o site fica dentro de uma subpasta com o nome do repositório (`/FlopPlus/`). Quando um link começa com barra (`/`), o navegador vai para a raiz do domínio (`username.github.io/index.html`) em vez da pasta do projeto, gerando erro 404.

Problema encontrado: Identificamos exatamente 5 links com barras absolutas espalhados em `index.html`, `cadastro.html`, `inicio.html` e `login.html`.

Alteração realizada: Corrigimos todos os 5 links para usar caminhos relativos (`../index.html`, `aboutme.html`, `HTML/aboutme.html`), tornando o site 100% portável.

Interação 3 — Criando um teste real para site estático

Prompt utilizado: *"Como o projeto é só HTML e CSS puro, o professor pediu uma etapa de testes e validação na pipeline de CI. O que eu posso testar de verdade num site estático pra não ficar só um comando vazio?"*

Resultado obtido: A IA sugeriu criar um script em Node.js que percorre os arquivos HTML e confere se todas as imagens, arquivos de CSS e páginas linkadas realmente existem no disco, além de validar a estrutura básica das tags (DOCTYPE, title, body) e barrar qualquer link absoluto que pudesse quebrar no deploy.

Problema encontrado: Nenhum; a ideia supriu exatamente a exigência de teste e validação do Lab.

Alteração realizada: Criamos o arquivo `tests/validate-integrity.js`, que executa 44 verificações automáticas com saída clara no terminal.

Interação 4 — Resolvendo erro de execução no Windows

Prompt utilizado: *"Fui rodar o htmlhint aqui no meu computador pelo terminal do VS Code e deu erro de módulo não encontrado. A pasta do meu projeto tá dentro de 'Instituto J&F', acho que o '&' tá atrapalhando o cmd. Como arrumo o package.json pra rodar sem dar erro aqui no Windows e também funcionar no GitHub Actions?"*

Resultado obtido: A IA explicou que o caractere `&` é interpretado pelo `cmd.exe` do Windows como separador de comandos, quebrando o script `.cmd` gerado pelo npm dentro da pasta `node_modules/.bin`.

Problema encontrado: Falha de execução do comando no ambiente de desenvolvimento local do Windows.

Alteração realizada: Configuramos o script `lint:ci` chamando diretamente o arquivo JavaScript via Node (`node node_modules/htmlhint/bin/htmlhint ...`), rodando perfeitamente tanto no Windows local quanto no runner Linux do GitHub Actions.

Interação 5 — Montando o arquivo final do GitHub Actions

Prompt utilizado: *"Beleza, os testes tão passando. Agora monta o arquivo ci-cd.yml pra mim? Preciso que o teste rode sempre que tiver push ou pull request pra main, e o deploy no GitHub Pages só aconteça se os testes passarem e for direto na main."*

Resultado obtido: A IA entregou o arquivo `.github/workflows/ci-cd.yml` estruturado com `needs: continuous-integration`, a condicional `if: github.ref == 'refs/heads/main' && github.event_name == 'push'`, permissões de OIDC e controle de concorrência.

Problema encontrado: Nenhum; a estrutura seguiu rigorosamente a especificação do Lab.

Alteração realizada: Adicionamos o workflow ao projeto e comentários explicativos em cada etapa.

Interação 6 — Criação de branch, .gitignore e push no Git

Prompt utilizado: *"Como eu crio uma branch nova chamada feature/ci-cd-pipeline, adiciono um .gitignore pra não subir as pastas do WebStorm e mando essa branch pro meu GitHub?"*

Resultado obtido: A IA gerou o arquivo `.gitignore` ignorando `.idea/`, `node_modules/` e logs, e forneceu os comandos do Git para criar a branch, comitar as alterações e fazer o push para o repositório remoto.

Problema encontrado: Havia arquivos da pasta `.idea/` soltos como untracked.

Alteração realizada: Criamos o `.gitignore`, executamos `git checkout -b feature/ci-cd-pipeline`, fizemos o commit das alterações e enviamos com sucesso com `git push -u origin feature/ci-cd-pipeline`.

5. Respostas aos Questionamentos Finais do Lab

Item Avaliado	Resposta do Aluno
1. Quantos prompts foram necessários para chegar à versão final?	Foram necessários **6 prompts** diretos e objetivos: (1) concepção da arquitetura dos jobs; (2) identificação e correção de rotas para o GitHub Pages; (3) elaboração do teste de integridade em Node.js; (4) resolução do bug de caminho com caractere especial no Windows; (5) elaboração do arquivo final `ci-cd.yml`; e (6) criação da branch, configuração do `.gitignore` e envio via Git.
2. Quais foram os principais problemas encontrados?	1. **Ausência original de testes:** O projeto era composto apenas por arquivos estáticos soltos, sem ferramentas de teste nem `package.json`; 2. **Links absolutos no HTML:** Haviam 5 referências usando barra na raiz (ex: `/index.html`), que quebram no GitHub Pages por este rodar em subdiretório (`/FlopPlus/`); 3. **Conflito do caractere '&' no Windows:** A pasta do projeto continha o nome "Instituto J&F", o que fazia o `cmd.exe` quebrar ao rodar executáveis `.cmd` gerados pelo npm; 4. **Arquivos de IDE não ignorados:** A pasta `.idea/` estava solta no repositório sem `.gitignore`.
3. Quais alterações foram necessárias?	• Criação do workflow `.github/workflows/ci-cd.yml` com os jobs de CI e CD; • Criação do `package.json` com os scripts `test`, `lint:ci` e dependência do `htmlhint`; • Desenvolvimento do script `tests/validate-integrity.js` com 44 checagens automáticas; • Correção dos 5 links absolutos nos arquivos HTML para caminhos relativos; • Criação do arquivo `.gitignore`.
4. Quais alterações foram realizadas manualmente?	• Navegação manual no navegador pelas páginas modificadas para validar se o layout, botões e imagens continuavam funcionando perfeitamente; • Conferência da autenticação das credenciais do Git no Windows para viabilizar o push remoto; • Definição do nome padronizado da branch de trabalho (`feature/ci-cd-pipeline`).

Item Avaliado	Resposta do Aluno
5. Como você verificou que a pipeline estava funcionando corretamente?	1. **Execução local dos testes de CI:** Rodagem de `npm test` (44 verificações de integridade aprovadas) e `npm run lint:ci` (5 arquivos HTML sem erros); 2. **Validação estrutural do YAML:** Verificação dos gatilhos, dependência `needs: continuous-integration` e condicional de branch `main`; 3. **Confirmação de Push Remoto:** Sucesso no envio da branch `feature/ci-cd-pipeline` para o repositório remoto com link de Pull Request gerado pelo GitHub; 4. **Conferência do fluxo CI → CD:** Garantia de que o deploy só ocorre automaticamente se a CI for concluída com êxito na branch principal.

6. Breve Análise do Processo de Desenvolvimento com IA

O apoio da ferramenta de IA Generativa durante a atividade permitiu acelerar substancialmente a criação das configurações da pipeline e proporcionou um aprendizado prático e preventivo. Ao invés de simplesmente gerar um código genérico, as perguntas direcionadas permitiram antecipar problemas clássicos de hospedagem estática, como a quebra de links absolutos no GitHub Pages, e viabilizaram a criação de uma suíte de testes customizada e realista para um projeto front-end.

Ao mesmo tempo, a atividade evidenciou a relevância fundamental do senso crítico do desenvolvedor: erros locais decorrentes de características do sistema operacional (como a colisão do caractere especial `&` no terminal do Windows) exigiram que o problema fosse analisado, testado e adaptado para que a solução final fosse verdadeiramente portável. A IA atuou como uma excelente assistente técnica de pair programming, cabendo ao desenvolvedor conduzir, testar e homologar os resultados.

7. Evidências da Execução Correta da Pipeline CI/CD

Evidência 1: Sucesso nos Testes Locais de CI (npm test & npm run lint:ci)

```
> flopplus@1.0.0 test
> node tests/validate-integrity.js

==================================================
  FLOP+ - TESTE DE INTEGRIDADE E VALIDACAO DE ASSETS
==================================================

[1/3] Verificando arquivos essenciais da aplicacao...
  ✓ index.html encontrado.
  ✓ LICENSE encontrado.
  ✓ CSS\style.css encontrado.
  ✓ IMG\logo.png encontrado.
  ✓ IMG\background.avif encontrado.
  ✓ index.html encontrado.
  ✓ HTML\aboutme.html encontrado.
  ✓ HTML\cadastro.html encontrado.
  ✓ HTML\inicio.html encontrado.
  ✓ HTML\login.html encontrado.

[2/3] Verificando estrutura e doctype dos arquivos HTML...
  ✓ Estrutura basica de index.html validada.
  ✓ Estrutura basica de HTML\aboutme.html validada.
  ✓ Estrutura basica de HTML\cadastro.html validada.
  ✓ Estrutura basica de HTML\inicio.html validada.
  ✓ Estrutura basica de HTML\login.html validada.

[3/3] Validando links (href) e assets (src)...

==================================================
TOTAL DE VERIFICACOES REALIZADAS: 44
SUCESSO: Todos os arquivos, estruturas e links estao integros!
==================================================

> flopplus@1.0.0 lint:ci
> node node_modules/htmlhint/bin/htmlhint index.html HTML/*.html

Scanned 5 files, no errors found (28 ms).
```

Evidência 2: Criação da Branch e Push Remoto para o GitHub

```
$ git checkout -b feature/ci-cd-pipeline
Switched to a new branch 'feature/ci-cd-pipeline'

$ git add .gitignore package.json package-lock.json tests/ .github/ HTML/ index.html
$ git commit -m "feat: implementa pipeline de CI/CD com GitHub Actions e GitHub Pages"
[feature/ci-cd-pipeline b9f6f7a] feat: implementa pipeline de CI/CD com GitHub Actions e GitHub P
ages
 9 files changed, 605 insertions(+), 5 deletions(-)

$ git push -u origin feature/ci-cd-pipeline
branch 'feature/ci-cd-pipeline' set up to track 'origin/feature/ci-cd-pipeline'.
To https://github.com/noecarrion/FlopPlus.git
 * [new branch]      feature/ci-cd-pipeline -> feature/ci-cd-pipeline
```

Instruções para Demonstração e Publicação no GitHub

1. No repositório do projeto (github.com/noecarrion/FlopPlus), acesse **Settings > Pages**;
2. Na opção **Build and deployment > Source**, selecione **"GitHub Actions"**;
3. Abra o **Pull Request** da branch `feature/ci-cd-pipeline` para a branch `main` pelo link: Criar Pull Request;
4. Na aba **Actions**, acompanhe a execução do job de validação (CI);
5. Ao aprovar e realizar o merge para a branch `main`, o job de CD será acionado automaticamente e publicará o site na URL: https://noecarrion.github.io/FlopPlus/.
